

第 1 章 项目概述	4
1.1 系统背景	4
1.2 四实体关系	4
1.3 技术栈.....	4
第 2 章 系统架构设计	6
2.1 三层架构	6
2.2 统一调度机制	6
2.3 一个 Tick 的完整数据流	6
2.4 命令总线	7
第 3 章 实体模型设计	8
3.1 Target 目标运动体	8
3.1.1 概述	8
3.1.2 五种运动模式	8
3.1.3 派生属性.....	8
3.1.4 配置参数.....	8
3.2 Gimbal 两轴云台	9
3.2.1 概述	9
3.2.2 电源状态机	9
3.2.3 串级 PID 控制器	9
3.2.4 被控对象模型	9
3.2.5 PID 参数总表	10
3.2.6 GimbalState 字段	10
3.3 Camera 相机	10
3.3.1 概述	10
3.3.2 针孔成像模型	10
3.3.3 变焦控制.....	11
3.3.4 成像参数.....	11
3.3.5 信标检测.....	11
3.4 Raspi 树莓派控制器	11
3.4.1 概述	12
3.4.2 三级延时管线	12
3.4.3 控制程序协议	12
3.4.4 基线跟踪程序	12
第 4 章 配置系统设计	14
4.1 配置结构	14

4.2 MOTION_MODE_PARAMS 注册表	14
4.3 添加新运动模式 (4 步)	14
4.4 派生属性	15
第 5 章 控制程序开发	16
5.1 ControlProgram 协议	16
5.2 obs 观测字典结构.....	16
5.3 Command 命令类型	16
5.4 自定义控制程序模板.....	16
5.5 CLI 注入	17
第 6 章 仿真运行与工具	18
6.1 CLI 参数	18
6.2 典型运行场景	18
6.3 GUI 实时仪表盘.....	18
6.4 可视化与调试工具.....	19
第 7 章 测试与验证	20
7.1 测试体系总览	20
7.2 测试覆盖范围	20
7.3 运行命令	20
7.4 通过标准	21
第 8 章 代码架构与规范	22
8.1 目录结构	22
8.2 设计模式	22
8.3 命名约定	23
8.4 维护约定	23
附录	24
A. 核心数据类型	24
B. 数值参数汇总	24

云台数字孪生仿真系统

技术文档 v1.0

Gimbal Digital Twin Simulation System

2026 年 4 月

第 1 章 项目概述

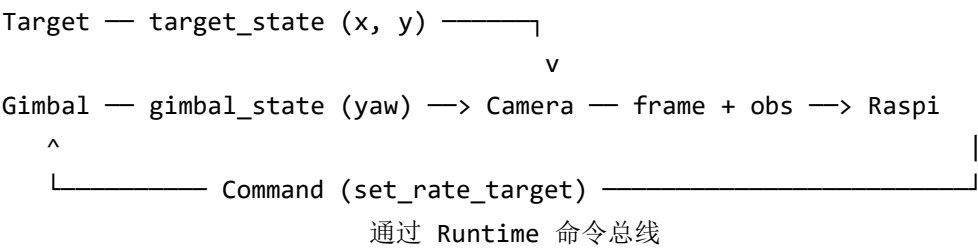
1.1 系统背景

云台数字孪生仿真系统是一个面向视觉伺服跟踪场景的实时仿真平台，用于在纯软件环境中复现「云台-相机-目标」闭环跟踪链路。系统通过数字孪生技术模拟真实硬件的行为，使开发者能够在无需物理设备的情况下进行控制算法设计、参数调优和性能评估。

1.2 四实体关系

系统由四个核心实体构成，形成完整的跟踪闭环：

- **Target（目标运动体）**：输出世界坐标 (x, y)，模拟被跟踪目标的运动
- **Gimbal（两轴云台）**：执行角速度/角度控制，输出姿态 yaw/pitch
- **Camera（相机）**：挂载在云台上，根据目标方位和云台姿态成像
- **Raspi（树莓派控制器）**：从帧中检测目标，输出控制命令，驱动云台转动



1.3 技术栈

组件	用途
Python 3.10+	核心开发语言，dataclass 类型系统
NumPy	矩阵运算、环形缓冲区
PyQt5	GUI 框架（仪表盘、配置编辑器、3D 可视化）
pyqtgraph 0.14	高性能实时绘图
matplotlib	2D/3D 科学可视化
unittest	单元测试与集成测试

第 2 章 系统架构设计

2.1 三层架构

系统采用三层架构，职责清晰分离：

- **实体层 (entities/)**：四个独立实体，各自包含模型(model)、控制(control)、客户端(client)和测试(tests)
- **运行时层 (runtime/)**：DigitalTwinRuntime 统一调度，管理命令总线、tick 推进和快照发布
- **应用层 (simulation/)**：CLI 参数解析、无头运行、GUI 仪表盘、Bootstrap 编排

2.2 统一调度机制

所有实体由 DigitalTwinRuntime 统一调度，每个 tick 按固定顺序推进：

收命令 -> Target.update -> Gimbal.update -> Camera.update -> Raspi.update -> 发布快照

命令仲裁采用 latest-wins 策略：同一 tick 内多条命令到同一实体时，最后一条生效。设备未 READY 时命令被拒绝，返回 CommandResult(accepted=False)。

2.3 一个 Tick 的完整数据流

- 1. _apply_due_commands()：分派到期命令到对应实体
- 2. target.update(dt, t)：推进运动模型，输出 TargetState{x_m, y_m, bearing_deg, distance_m}
- 3. gimbal.update(dt, t)：执行串级 PID，驱动被控对象，输出 GimbalState
- 4. camera.update(dt, t, target, gimbal)：计算方位角，渲染帧，输出 CameraState + FramePacket
- 5. raspi.update(t, world_obs, ...)：延时管线处理，调用控制程序，输出 Command

- 6. 发布 WorldSnapshot: 包含全部实体状态, 供 UI 和工具读取

2.4 命令总线

命令通过三级路径完成闭环:

```
Raspi.on_tick(obs) → cmds → DelayPipeline → submit_cmd →  
Runtime._pending_commands
```

下一 tick 的 `_apply_due_commands()` 将到期命令分派到对应实体。

典型闭环路径 (基线跟踪): 帧中检测目标质心 → 计算像素误差 ($u - cx$) → 比例映射 $yaw_rate = Kp * pixel_error$ → `Command(gimbal, set_rate_target)` → Gimbal 转动 → Camera 帧变化 → 循环。

第 3 章 实体模型设计

3.1 Target 目标运动体

3.1.1 概述

Target 是只读的被动实体——不接受任何命令，从创建起即活跃，无电源状态机。其他实体依赖 target_state 中的 x_m , y_m 计算方位角。

3.1.2 五种运动模式

核心方法 TargetKinematics2D.step(dt) -> (x, y)，根据 motion_type 分支：

模式	公式	使用参数
constant_velocity	$x += vx*dt, y += vy*dt$	velocity_x/y_mps
constant_accel	$v += a*dt, x += v*dt$	velocity + accel_x/y_mps2
sinusoidal	$x = x_0, y = A*\sin(2*\pi*f*t)$	sin_amplitude/frequency
random_walk	$v = v*damp + a*dt, x += v*dt$	random_max_accel/damping/seed
waypoint	朝航点飞行，到达后切换	waypoints, arrival_radius

3.1.3 派生属性

```
bearing_deg = atan2(y, x) 转角度    // 目标方位角
distance_m  = hypot(x, y)   // 目标距离
```

3.1.4 配置参数

参数	默认值	含义
motion_type	"sinusoidal"	运动模式 (Literal 类型, 5 选 1)
initial_x_m	100.0	初始 X 坐标 (米)
initial_y_m	0.0	初始 Y 坐标 (米)
velocity_x/y_mps	0.0 / 1.5	初速度 (m/s)
accel_x/y_mps2	0.0 / 0.3	加速度 (m/s^2)
sin_amplitude_m	15.0	正弦振幅 (米)
sin_frequency_hz	0.2	正弦频率 (Hz)

random_max_accel_mps2	1.0	随机最大加速度
random_damping	0.98	速度阻尼系数
random_seed	42	随机种子（可复现）
waypoints	None	航点列表 [(x, y, speed), ...]
waypoint_arrival_radius_m	1.0	到达判定半径（米）

3.2 Gimbal 两轴云台

3.2.1 概述

Gimbal 是系统的核心被控对象，接收角度/角速度指令，通过串级 PID 控制器驱动一阶惯性被控对象。

3.2.2 电源状态机

OFF $\xrightarrow{\text{(power_on)}}$ BOOTING (1.5s) $\xrightarrow{\quad}$ READY

READY $\xrightarrow{\text{(power_off)}}$ OFF

非 READY 状态拒绝所有控制命令

3.2.3 串级 PID 控制器

采用双环串级结构：外环角度环（P-only, 50Hz）+ 内环角速度环（PI, 200Hz）：

环节	频率	算法	参数
角度外环	50 Hz (dt=20ms)	P-only	angle_kp = 14.0
角速度内环	200 Hz (dt=5ms)	PI	rate_kp=1.6, rate_ki=5.0
积分限幅	-	Anti-windup	integral_limit = 30.0
输出限幅	-	Clamp	actuator_limit = 60.0 dps

外环角度误差计算采用 wrap_pm180 归一化到 [-180, 180)，输出角速度参考值并限幅到 max_rate。内环对角速度误差做 PI 运算，积分项带 anti-windup 限幅。

3.2.4 被控对象模型

一阶惯性环节，模拟电机响应延迟：

```
alpha = dt / (tau + dt)           // tau = 0.03s
rate = (1 - alpha) * current + alpha * cmd
```

```
position += rate * dt           // 积分得到角度
```

Yaw 轴无界（仅在显示时归一化），Pitch 轴硬限位 $[-135, 90]$ ，到达限位时对应方向速率归零。

3.2.5 PID 参数总表

参数	Yaw	Pitch	说明
angle_kp	14.0	14.0	外环比例增益
rate_kp	1.6	1.6	内环比例增益
rate_ki	5.0	5.0	内环积分增益
rate_integral_limit	30.0	30.0	积分限幅
actuator_cmd_limit	60.0 dps	60.0 dps	输出限幅
max_rate	60.0 dps	60.0 dps	外环参考限幅
response_tau	0.03s	0.03s	被控对象时间常数

3.2.6 GimbalState 字段

字段	说明
yaw_deg_internal	内部连续角度（无界）
yaw_deg_display	显示角度（0-360 或 -180~180，由 config 决定）
pitch_deg	俯仰角（硬限位 $[-135, 90]$ ）
yaw_rate_dps / pitch_rate_dps	实际角速度
yaw_rate_ref_dps / pitch_rate_ref_dps	角速度参考值（外环输出）
mode	ANGLE_MODE / RATE_MODE
power_state	OFF / BOOTING / READY

3.3 Camera 相机

3.3.1 概述

Camera 挂载在云台上，根据目标方位和云台姿态进行针孔成像。输出 CameraState 和 FramePacket（含灰度图像与内参）。

3.3.2 针孔成像模型

目标方位角 α 的计算：

```
bearing = atan2(target.y, target.x)           // 目标方位角
yaw = radians(gimbal.yaw_deg_internal)        // 云台航向角
alpha = (bearing - yaw + pi) % (2*pi) - pi     // 归一化到 [-pi, pi]
```

像素坐标映射：

```
u = f_px * tan(alpha) + w/2                  // 水平像素坐标
v = h/2                                       // 垂直居中（1D 简化）
in_fov = |alpha| <= fov_half                 // 是否在视野内
```

信标渲染为 2D 高斯光斑 ($\sigma = 3.2\text{ px}$)，叠加高斯噪声 ($\text{std} = 0.5$)。

3.3.3 变焦控制

ZoomController 采用一阶惯性模型 ($\tau = 0.2\text{s}$)，支持两种模式：

- 目标焦距模式：指数趋近 f_{target}
- 恒速模式： $f += \text{clamp}(\text{rate}, -120, +120) * dt$

焦距范围：[4.4mm, 200mm]。

3.3.4 成像参数

参数	值	说明
分辨率	640 x 480	像素
传感器尺寸	4.8 x 3.6 mm	物理尺寸
默认焦距	12.0 mm	$f_{\text{px}} = 1600$
FOV (12mm)	22.6 deg	水平全视角
pixel_size	7.5 um	4.8/640
px_per_deg (12mm)	27.9 px/deg	$1600 * \pi / 180$
信标检测阈值	180	灰度值

3.3.5 信标检测

`detect_beacon_centroid(image, threshold=180)`：查找灰度 $\geq \text{threshold}$ 的像素，计算质心 (cx, cy) 和置信度。返回 `Detection(found, cx, cy, confidence)`。

3.4 Raspi 树莓派控制器

3.4.1 概述

Raspi 是闭环控制的核心节点：接收观测，运行控制程序，输出命令。通过三级延时管线模拟真实硬件延迟。

3.4.2 三级延时管线

DelayPipeline 使用三个最小堆实现三级延迟：

阶段	默认延迟	说明
观测获取	image_read + state_read delay	从 Runtime 读取观测的延迟
图像处理	image_process_delay = 20ms	控制程序处理观测的延迟
命令发送	command_tx_delay	命令传送到 Runtime 的延迟

每级可独立配置延迟和抖动（高斯噪声，单侧 $\max(0, \text{gauss}(0, \text{std}))$ ）。每帧的观测经过管线后才会被控制程序处理。

3.4.3 控制程序协议

```
class ControlProgram(Protocol):
    def on_tick(self, obs: dict) -> list[Command]: ...
```

obs 包含 timestamp, target, gimbal, camera, frame。返回 list[Command] 控制设备。

3.4.4 基线跟踪程序

BaselineTrackerProgram 实现简单的比例控制：

```
pixel_error = det.cx - cx
yaw_rate = clamp(kp * pixel_error, -max_rate, +max_rate) // kp=0.08 dps/px
若 |pixel_error| < deadband(2px), rate = 0
若目标丢失, rate = lost_target_hold_rate(0)
```

参数	默认值	说明
yaw_rate_kp	0.08 dps/px	比例增益
max_yaw_rate	60.0 dps	角速度限幅
deadband	2.0 px	死区
lost_target_hold	0.0 dps	丢失目标保持速率

第 4 章 配置系统设计

4.1 配置结构

config.py 包含 10 个 dataclass 配置类，均为模块级单例：

配置类	实例名	覆盖范围
CameraConfig	camera_cfg	分辨率、传感器尺寸、焦距
GimbalConfig	gimbal_cfg	角度范围、响应时间常数
AxisLimitConfig	axis_limit_cfg	Pitch 限位、最大角速度
LoopConfig	loop_cfg	内外环频率
ControlPreset	control_preset_cfg	PID 参数
YawDisplayConfig	yaw_display_cfg	航向角显示模式
RaspiConfig	raspi_cfg	启动延时
RaspiDelayConfig	raspi_delay_cfg	三级延时参数
TargetConfig	target_cfg	运动模式与参数
SceneConfig	scene_cfg	场景、动画、绘图参数

4.2 MOTION_MODE_PARAMS 注册表

为支持运动模式的可扩展性，config.py 定义了 MOTION_MODE_PARAMS 注册表：

```
MOTION_MODE_PARAMS = {
    "sinusoidal": ["sin_amplitude_m", "sin_frequency_hz"],
    "constant_velocity": ["velocity_x_mps", "velocity_y_mps"],
    "constant_accel": [... , "accel_x_mps2", "accel_y_mps2"],
    "random_walk": ["random_max_accel_mps2", "random_damping", ...],
    "waypoint": ["waypoints", "waypoint_arrival_radius_m"],
}
```

Config Editor 自动读取此注册表，切换 motion_type 下拉框时隐藏不属于当前模式的参数。

4.3 添加新运动模式（4 步）

- 1. 在 TargetConfig.motion_type 的 Literal[...] 中加模式名

- 2. 在 MOTION_MODE_PARAMS 中加模式 -> 字段映射
- 3. 在 TargetConfig 中加新参数字段
- 4. 在 TargetKinematics2D.step() 中加 elif 分支实现物理逻辑

Config Editor 自动读取 Literal 类型生成下拉选项，并根据 MOTION_MODE_PARAMS 过滤显示对应参数。

4.4 派生属性

CameraConfig 定义了三个 @property 派生属性，在配置变更时自动重新计算：

- $\text{pixel_size_mm} = \text{sensor_w_mm} / \text{resolution_w} = 0.0075 \text{ mm}$
- $\text{focal_length_px} = \text{focal_length_mm} / \text{pixel_size_mm} = 1600 \text{ px}$
- $\text{fov_h_deg} = 2 * \text{atan}(\text{sensor_w_mm} / (2 * \text{focal_length_mm})) = 22.6 \text{ deg}$

第 5 章 控制程序开发

5.1 ControlProgram 协议

```
from runtime.types import Command

class ControlProgram(Protocol):
    def on_tick(self, obs: dict) -> list[Command]: ...
```

5.2 obs 观测字典结构

键	内容	类型
timestamp	当前仿真时间	float
target	{x_m, y_m, bearing_deg, distance_m}	dict
gimbal	{power_state, mode, yaw_deg_internal, yaw_deg_display, pitch_deg, ...}	dict
camera	{power_state, f_current_mm, frame_id, in_fov, u_px, v_px}	dict
frame	FramePacket(image, intrinsics, optional_gt)	FramePacket

5.3 Command 命令类型

target	合法 action
gimbal	power_on, power_off, set_mode, set_angle_target, set_rate_target
camera	power_on, power_off, set_zoom_target_mm, zoom_by, set_zoom_rate_mmpps
raspi	power_on, power_off

5.4 自定义控制程序模板

```
from runtime.types import Command
from entities.camera.entity import detect_beacon_centroid

class MyTracker:
    def __init__(self, kp=0.1):
        self.kp = kp
```



```
def on_tick(self, obs: dict) -> list[Command]:
    ts = float(obs['timestamp'])
    cmds = []
    # 1. 确保 RATE_MODE
    # 2. 从 frame 检测目标
    # 3. 计算像素误差, 比例映射
    # 4. 返回 Command 列表
    return cmds
```

5.5 CLI 注入

命令行注入: 格式 module:Class

```
python app.py --control-program my_tracker:MyTracker --duration 10
```

代码注入:

```
from simulation.bootstrap import build_runtime
runtime = build_runtime(control_program=MyTracker())
```

第 6 章 仿真运行与工具

6.1 CLI 参数

参数	类型	默认值	说明
--duration	float	60.0	运行时长 (秒)
--mode	realtime offline	"realtime"	运行模式
--delay-ms	float	0.0	Raspi 链路延时 (毫秒)
--no-gui	flag	False	不启用窗口, 仅控制台输出
--control-program	str	""	控制程序 module:Class
--target-type	str	""	目标运动类型
--waypoints	str	""	航点 "(x,y,s),(x,y,s)"

6.2 典型运行场景

无 GUI 快速验证

```
python app.py --no-gui --mode offline --duration 1.0
```

GUI 实时仿真

```
python app.py --mode realtime --duration 60
```

带延时链路

```
python app.py --mode realtime --duration 30 --delay-ms 20
```

自定义控制程序

```
python app.py --no-gui --control-program my_tracker:MyTracker --duration 5
```

航点轨迹

```
python app.py --no-gui --waypoints "(100,0,2),(80,30,1.5)" --duration 20
```

随机运动 + 延时

```
python app.py --no-gui --target-type random_walk --delay-ms 15 --duration 20
```

6.3 GUI 实时仪表盘

仪表盘界面布局 (1680x980) :

- 左侧上方：世界视图（轨迹、云台指向、FOV 扇形）
- 左侧下方：时间轴曲线（像素误差、角速度参考、角度误差）
- 右侧上方：双视角对比（相机原始帧 vs Raspi 延时观测帧）
- 右侧下方：Tab 信息区（核心状态 / 诊断信息）
- 控制栏：开始、暂停、重置、保存快照、链路延时设置

6.4 可视化与调试工具

工具	调用命令	说明
目标轨迹预览	python tools/target_preview.py	2D 动画预览目标运动轨迹
3D 相机投影	python tools/camera_3d_viewer.py	交互式 3D 针孔模型可视化
阶跃响应分析	python tools/step_response.py	实时云台控制工作台，分析阶跃响应
配置编辑器	python tools/config_editor.py	实体导航式 GUI 配置编辑
数据录制	python -m tools.record_session	仿真运行导出 CSV
离线回放	python -m tools.replay_session	CSV 回放驱动控制程序

第 7 章 测试与验证

7.1 测试体系总览

项目包含 228 个单元测试 + 4 个集成测试，总计 232 个测试方法：

实体 / 层	测试文件	测试数
Target	entities/target/tests/test_target_entity.py	64
Gimbal	entities/gimbal/tests/test_gimbal_entity.py	63
Camera	entities/camera/tests/test_camera_entity.py	67
Raspi	entities/raspi/tests/test_raspi_entity.py	26
Tracker	entities/raspi/tests/test_tracker_program.py	2
集成	tests/test_gimbal_2axis_core.py	2
集成	tests/test_digital_twin_runtime.py	2
集成	tests/test_runtime_api.py	1
集成	tests/test_pid_tuner_smoke.py	1
合计		228

7.2 测试覆盖范围

- Target: 5 种运动模式 + bearing/distance + Entity 包装 + 边界条件
- Gimbal: 电源状态机 + NOT_READY 拒绝 + ANGLE/RATE 模式 + pitch 限位 + yaw wrap + 一阶响应
- Camera: 电源状态机 + zoom 控制 + FOV 检查 + 像素映射 + 质心检测 + 帧生成
- Raspi: 电源状态机 + 控制程序加载 + 零延时/有延时管线 + backlog + delay profile

7.3 运行命令

```
# 全部实体测试
python -m unittest entities.target.tests.test_target_entity \
    entities.gimbal.tests.test_gimbal_entity \
```

```
entities.camera.tests.test_camera_entity \  
entities.raspi.tests.test_raspi_entity -v
```

主线回归测试

```
python -m unittest discover -s tests -v
```

组装冒烟测试

```
python app.py --no-gui --mode offline --duration 1.0
```

7.4 通过标准

- 所有测试通过 (228 单元 + 4 集成 = 232 total)
- 无 GUI 冒烟输出连续、无异常终止
- 关键字段 (yaw/pitch/u/v/in_fov/backlog) 正常刷新

第 8 章 代码架构与规范

8.1 目录结构

```

systemSimulation/
+-- app.py                # 主入口（透传到 simulation.cli）
+-- config.py             # 统一配置（10 个 dataclass +
    MOTION_MODE_PARAMS）
+-- simulation/           # 应用编排层
|   +-- bootstrap.py      # build_runtime / start_stack
|   +-- cli.py            # 参数解析 + 入口分发
|   +-- headless.py       # 无 GUI 运行入口
|   +-- worker.py         # 仿真推进 QThread
|   +-- state_buffer.py   # UI 线程安全缓冲
|   +-- gui/
|       +-- window.py     # 仪表盘主窗口
|       +-- runner.py     # create_dashboard / run_gui
+-- runtime/
|   +-- digital_twin_runtime.py # 世界时钟 / 命令总线 / 调度器
|   +-- types.py          # Command / WorldSnapshot / POWER_*
|   +-- clients.py        # Client 导出
+-- entities/
|   +-- gimbal/           # entity / model / control / client /
tests
|   +-- camera/           # entity / model / control / client /
tests
|   +-- target/           # entity / model / client / tests
|   +-- raspi/            # entity / model / pipeline /
control_program
+-- tests/                # 主线回归测试
+-- tools/                # 可视化与调试工具
+-- docs/                 # 文档
+-- output/               # 运行产物

```

8.2 设计模式

模式	应用场景
dataclass 配置	config.py 中 10 个配置类，类型安全 + 自动补全
Protocol 接口	ControlProgram 协议，鸭子类型控制程序
Client 代理	GimbalClient / CameraClient / RaspiClient 封装命令提交
观察者回调	Raspi 通过 submit_cmd 回调注入命令到 Runtime

工厂方法	build_runtime() 一键创建并启动完整运行时
注册表模式	MOTION_MODE_PARAMS 映射运动模式到参数字段

8.3 命名约定

参数名使用物理量 + 单位后缀：

后缀	含义	示例
_s	秒 (seconds)	boot_delay_s, response_tau_s
_hz	赫兹 (Hz)	sin_frequency_hz, angle_loop_hz
_deg	度 (degrees)	pitch_min_deg, angle_max_deg
_dps	度/秒 (deg/s)	max_velocity_dps, yaw_rate_dps
_mm	毫米 (mm)	focal_length_mm, sensor_w_mm
_m	米 (meters)	initial_x_m, sin_amplitude_m
_mps	米/秒 (m/s)	velocity_x_mps
_mps2	米/秒 ²	accel_x_mps2
_px	像素 (pixels)	deadband_px

8.4 维护约定

- 新功能优先落在 entities/* 与 runtime/*
- config.py 仅保留主线配置
- 修改实体代码时，同步更新对应实体的 README.md
- 每次迭代更新 workspace_meta/agent_log.md

附录

A. 核心数据类型

类型	字段	说明
Command	target, action, payload, timestamp, source	控制命令
CommandResult	accepted, code, message	命令执行结果
Detection	found, cx, cy, confidence	目标检测结果
FramePacket	timestamp, image, intrinsics, optional_gt	相机帧数据
WorldSnapshot	timestamp, target, gimbal, camera, raspi	世界快照

B. 数值参数汇总

参数	值	来源
角度外环频率	50 Hz (dt=20ms)	LoopConfig
角速度内环频率	200 Hz (dt=5ms)	LoopConfig
被控对象时间常数	0.03 s	GimbalConfig
Gimbal 启动延时	1.5 s	GimbalEntity
Camera 启动延时	0.5 s	CameraEntity
Raspi 启动延时	1.0 s	RaspiConfig
默认图像处理延时	0.02 s	RaspiDelayConfig
变焦时间常数	0.2 s	ZoomController
变焦最大速率	120 mm/s	ZoomController
场景时间步长	0.005 s (200 Hz)	SceneConfig
跟踪 kp	0.08 dps/px	TrackerTuning
跟踪死区	2.0 px	TrackerTuning